# 📘 ExoHabit AI

ExoplanetHabitability Prediction System

**Milestone 3 : Backend API Integration & Frontend UI Development**

---

## 🔷 1. Introduction

ExoHabit AI is a**full-stack machine learning web application** designed to predict the habitability of exoplanets using planetary and stellar parameters. With the increasing discovery of exoplanets, identifying potentially habitable planets has become a crucial research area.
**This project integrates:**

- A trained **Machine Learning model**
- A **Flask-based backend API**
- A responsive **frontend user interface**

The system enables users to input planetary data and obtain real-time predictions along with confidence scores.

---

## 🔷 2. Objectives

| OBJECTIVE NO. | DESCRIPTION |
|---|---|
| 1 | Deploy ML model using Flask API |
| 2 | Build interactive frontend UI |
| 3 | Enable real-time prediction |
| 4 | Ensure proper validation and error handling |
| 5 | Achieve seamless frontend-backend integration |

# 🔷 3. System Architecture

## 🔹 Overall Flow

text
User → Frontend → Flask API → ML Model → Response → UI

## 🔹 Component Description

| COMPONENT | ROLE |
|---|---|
| Frontend | Collects user input and displays results |
| Backend (Flask) | Processes requests and handles API logic |
| ML Model | Predicts habitability |
| Database/Model File | Stores trained model |

---

# 🔷4. Module 5: Backend API Development

---

## 🔹 4.1 Backend Overview

The backend is developed using Flask and is responsible for handling user requests, processing data, and returning predictions.

---

## 🔹 4.2 Model Integration

| ASPECT | DESCRIPTION |
|---|---|
| Model Type | Random Forest Classifier |
| Storage | model.pkl file |
| Loading Method | joblib.load() |
| Benefit | Faster prediction, efficient processing |

---

## 🔹 4.3 API Endpoints

| ENDPOINT | METHOD | DESCRIPTION |
|---|---|---|
| / | GET | Check server status |
| /predict | POST | Predict habitability |
| /rank | POST | Rank multiple planets |

---

## 🔹 4.4 /predict Endpoint

### 📥 Request Format

| PARAMETER | TYPE | DESCRIPTION |
|---|---|---|
| pl_rade | float | Planet radius |
| st _lum | float | Star luminosity |
| pl_orbper | float | Orbital period |
| st _teff | float | Star temperature |
| pl_eqt | float | Planet temperature |

| PARAMETER | TYPE | DESCRIPTION |
|---|---|---|
| pl_orbsmax | float | Distance from star |

## 📌 Example Request

```
json
{
  "pl_rade": 1.2,
  "st_lum": 1.0,
  "pl_orbper": 365,
  "st_teff": 5778,
  "pl_eqt": 288,
  "pl_orbsmax": 1.0
}
```

---

## 📤 Response Format

| FIELD | DESCRIPTION |
|---|---|
| status | success/error |
| prediction | 1 (habitable) / 0 (not habitable) |
| habitability_score | Confidence score |
| message | Status message |

## 📌 Example Response

```
json
{
  "status": "success",
  "prediction": 1,
  "habitability_score": 0.87,
  "message": "Prediction successful"
}
```

---

## 🔹 4.5 /rank Endpoint

### 📥 Request Format

- Accepts a list of planets

### 📌 Example

```
json
[
   { "pl_rade": 1.2, "st_lum": 1.0, "pl_orbper": 365, "st_teff":
5778, "pl_eqt": 288, "pl_orbsmax": 1.0 },
   { "pl_rade": 2.5, "st_lum": 0.5, "pl_orbper": 100, "st_teff":
4500, "pl_eqt": 400, "pl_orbsmax": 0.3 }
]
```

---

### 📤 Response Format

| FIELD | DESCRIPTION |
|---|---|
| status | success/error |
| ranked_planets | List sorted by score |

---

## 🔹 4.6 Input Validation

| VALIDATION TYPE | DESCRIPTION |
|---|---|
| Missing Fields | Checks all required parameters |
| Data Type Check | Ensures numeric values |
| Empty Input | Handles null requests |

### 📌 Importance:

- Prevents runtime errors
- Ensures accurate predictions

- Improves system reliability

---

### 🔹 4.7 Error Handling

| ERROR TYPE | EXAMPLE MESSAGE |
|---|---|
| Missing field | Missing field: pl_rade |
| Invalid type | Invalid type for input |
| Empty input | No input data provided |

---

### 🔹 4.8 Backend Testing

| TOOL | PURPOSE |
|---|---|
| Browser | Test root endpoint |
| Postman | API testing |
| curl | Command-line testing |

---

# 🔷5. Module 6: Frontend UI Development

---

### 🔹 5.1 Overview

The frontend provides an interactive interface for users to input data and view results.

---

## 5.2 Technologies Used

| TECHNOLOGY | PURPOSE |
|---|---|
| HTML | Structure |
| CSS | Styling |
| Bootstrap | Responsive design |
| JavaScript | API communication |

## 5.3 UI Components

| COMPONENT | DESCRIPTION |
|---|---|
| Input Form | Collects user data |
| Submit Button | Sends request |
| Result Section | Displays prediction |
| Alerts | Shows errors |

## 5.4 Input Form Design

| FEATURE | DESCRIPTION |
|---|---|
| Labels | Clear field identification |
| Placeholders | User guidance |
| Required Fields | Prevent empty input |
| Numeric Input | Ensures correct data |

## 5.5 Frontend–Backend Integration

| ASPECT | DESCRIPTION |
|---|---|
| Method | Fetch API |
| Data Format | JSON |
| Type | POST request |
| Endpoint | /predict |

---

## 5.6 Asynchronous Handling

| FEATURE | BENEFIT |
|---|---|
| .then() | Handles success |
| .catch() | Handles errors |
| No reload | Better UX |

---

## 5.7 Result Display

| ELEMENT | PURPOSE |
|---|---|
| Cards | Show results |
| Alerts | Display errors |
| Text | Show prediction |

---

### 🔹 5.8 Error Handling

| ERROR TYPE | HANDLING |
|---|---|
| Invalid input | Alert message |
| Backend failure | Error message |
| Network issue | Retry message |

---

### 🔹 5.9 Responsiveness

| FEATURE | BENEFIT |
|---|---|
| Bootstrap Grid | Adaptive layout |
| Mobile Friendly | Works on phones |
| Clean UI | Better readability |

---

# 🔷 6. Integration Testing

### 🔹 System Flow

text
User Input → Frontend → Backend → ML Model → Response → UI

---

### 🔹 Test Cases

| TEST CASE | INPUT TYPE | EXPECTED OUTPUT | RESULT |
|---|---|---|---|
| Valid Input | Correct values | Prediction shown | Pass |

| TEST CASE | INPUT TYPE | EXPECTED OUTPUT | RESULT |
|---|---|---|---|
| Missing Field | Incomplete data | Error message | Pass |
| Wrong Type | String input | Validation error | Pass |

---

# 🔷 7. Conclusion

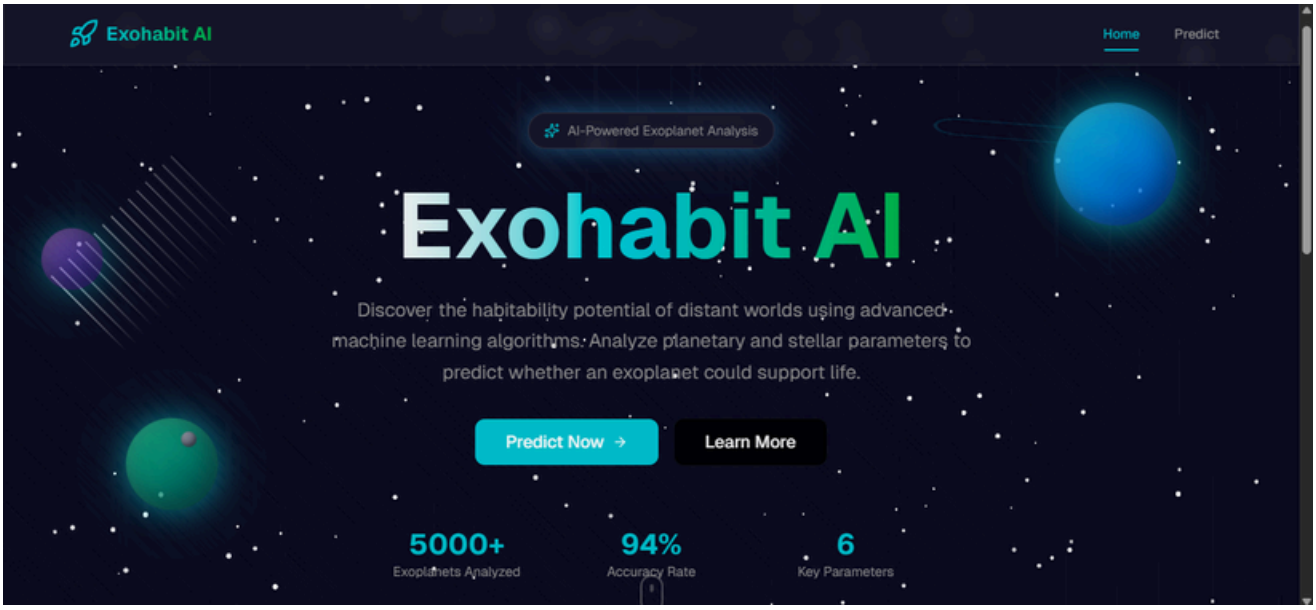
TheExoHabit AIsystemsuccessfullydemonstrates the integration of:

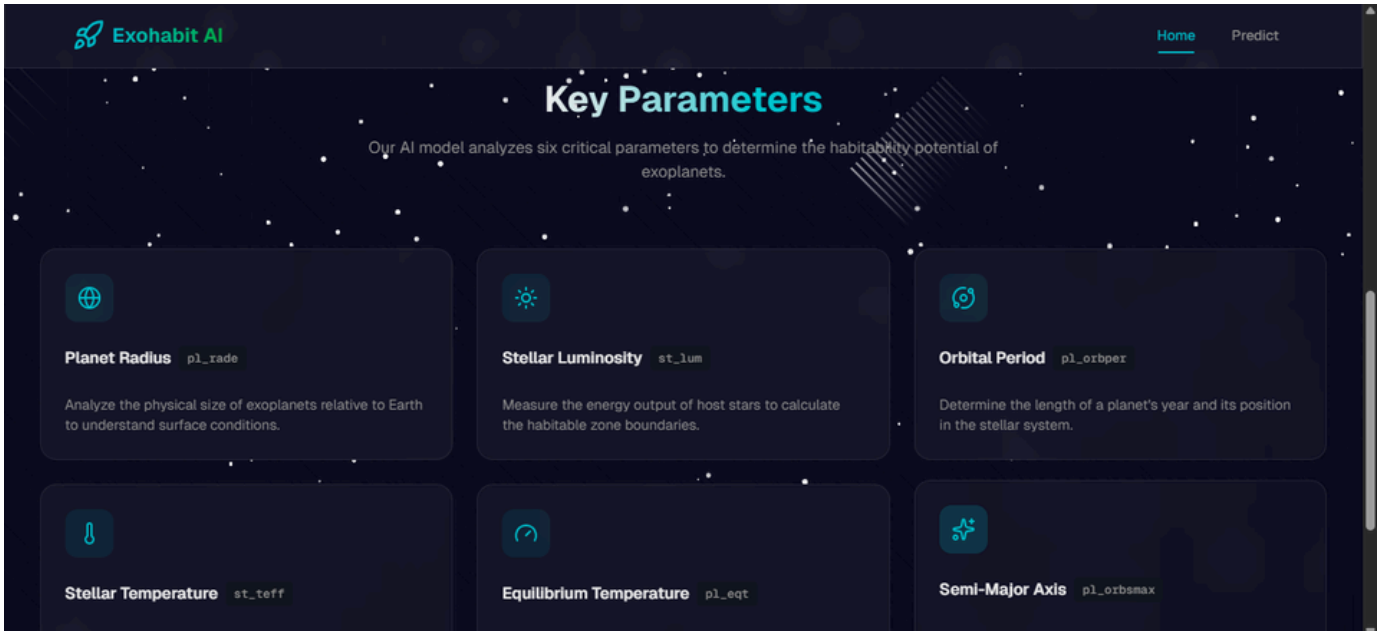
- Machine Learning
- Backend API
- Frontend UI

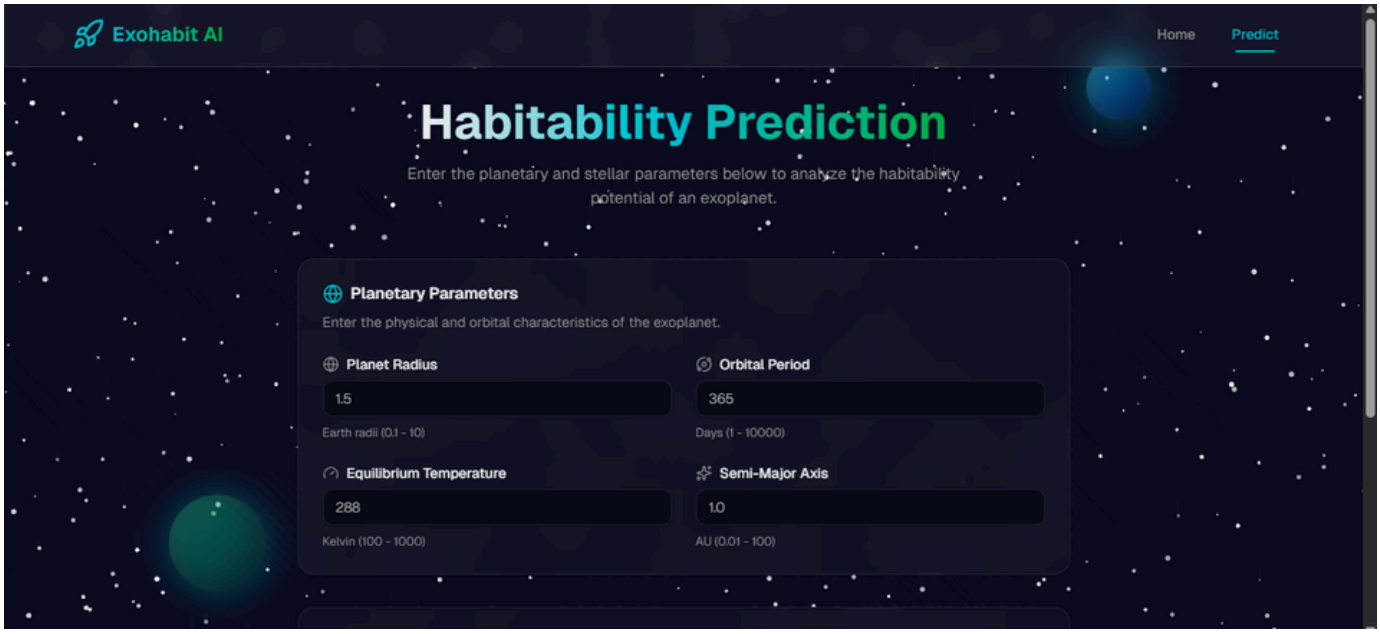
The application provides accurate predictions, ensures proper validation, and delivers a seamless user experience.

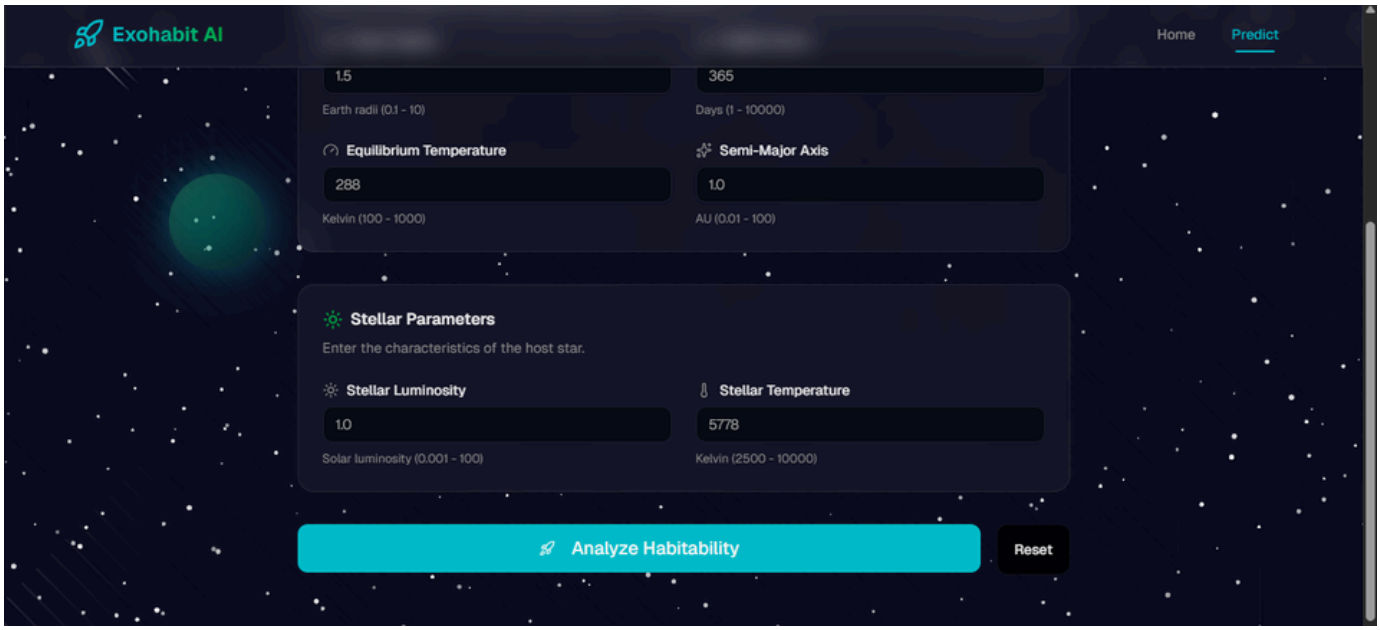
---

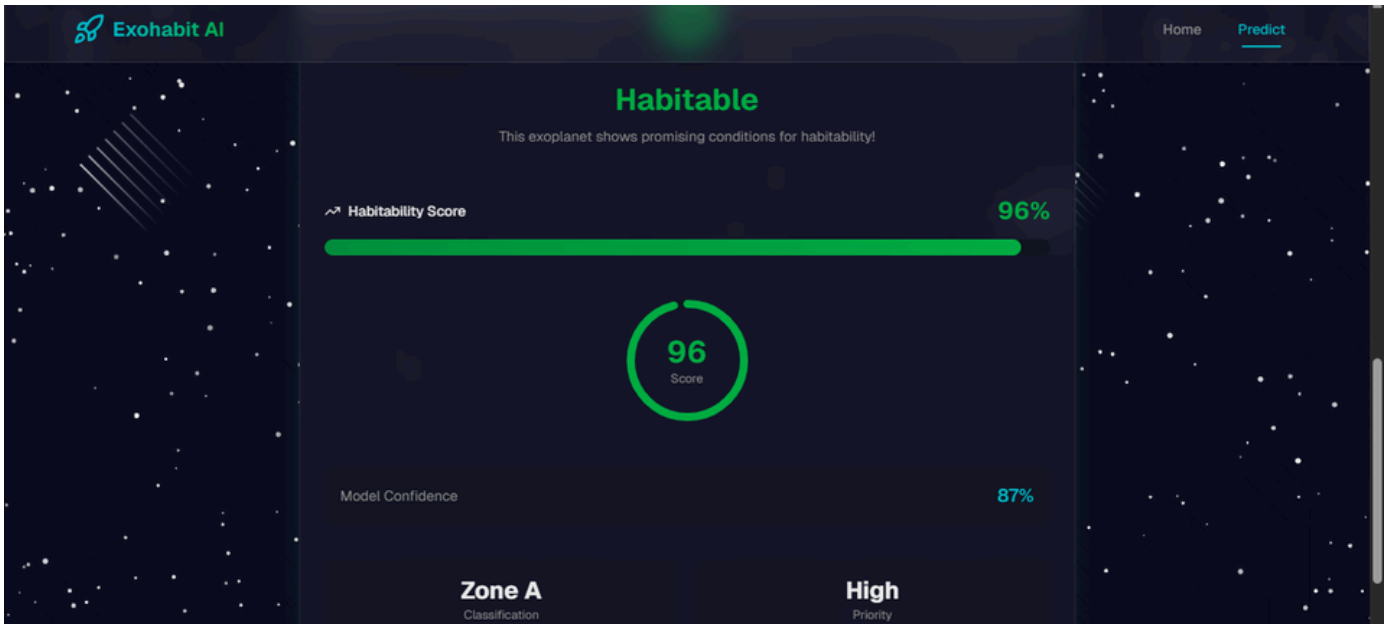
# 🔷 8. Screenshots




Exohabit AI
Home
Predict
Key Parameters
Our AI model analyzes six critical parameters to determine the habitability potential of exoplanets.
Planet Radius pl_rade
Analyze the physical size of exoplanets relative to Earth to understand surface conditions.
Stellar Luminosity st_lum
Measure the energy output of host stars to calculate the habitable zone boundaries.
Orbital Period pl_orbper
Determine the length of a planet's year and its position in the stellar system.
Stellar Temperature st_teff
Equilibrium Temperature pl_eqt
Semi-Major Axis pl_orbsmax


Exohabit AI
Home
Predict
Habitability Prediction
Enter the planetary and stellar parameters below to analyze the habitability potential of an exoplanet.
Planetary Parameters
Enter the physical and orbital characteristics of the exoplanet.
Planet Radius
1.5
Earth radii (0.1 - 10)
Orbital Period
365
Days (1 - 10000)
Equilibrium Temperature
288
Kelvin (100 - 1000)
Semi-Major Axis
1.0
AU (0.01 - 100)


Exohabit AI
Home
Predict
1.5
Earth radii (0.1 - 10)
365
Days (1 - 10000)
Equilibrium Temperature
288
Kelvin (100 - 1000)
Semi-Major Axis
1.0
AU (0.01 - 100)
Stellar Parameters
Enter the characteristics of the host star.
Stellar Luminosity
1.0
Solar luminosity (0.001 - 100)
Stellar Temperature
5778
Kelvin (2500 - 10000)
Analyze Habitability
Reset


Exohabit AI
Home
Predict
Habitable
This exoplanet shows promising conditions for habitability!
Habitability Score
96%
96
Score
Model Confidence
87%
Zone A
Classification
High
Priority